

1. Canonical event

Backend A JSON schema

```
{
  "timestamp": "2026-02-19T13:50:01Z",
  "episode_id": 23,
  "seed": 22260,
  "event_type": "process",
  "host": "ws-02",
  "user": "bob",
  "src_ip": "10.0.10.22",
  "dst_ip": null,
  "action": "process_start",
  "outcome": "success",
  "severity": "low",
  "process_name": "python",
  "tags": ["benign", "noisy_process"]
}
```

Backend A event is generated by the cyber-range / episode generator in the canonical telemetry format

same event

2. hard4 drift

Profile cycles across four schema variants:
hard4 selects/emits one of four possible Backend B schema variants per episode

v1_rename

Renamed fields

```
"when_utc": "2026-02-19T13:50:01Z"
"evt_kind": "process"
"origin_addr": "10.0.10.22"
"op_name": "process_start"
"risk_code": "low"
"labels_v2": ["benign", "noisy_process"]
```

v2_types

Renamed + retyped

```
"ts_epoch_ms": 1771509001000
"evt_type_name": "process"
"src_addr": "10.0.10.22"
"ok": true
"sev_level": 1
"tag_blob": "benign,noisy_process"
```

v3_semantic

Semantic reinterpretation

```
"tstamp": "2026-02-19T13:50:01Z"
"kind": "proc_event"
"ip_from": "10.0.10.22"
"verb": "proc_spawn"
"priority": "P3"
"labels": ["BENIGN", "NOISY_PROCESS"]
```

v4_sparse

Restructured + optional fields

```
"time_obs": "2026-02-19T13:50:01Z"
"cat": "process"
"src": "10.0.10.22"
"op_name_v2": "process_start"
"tagset": ["benign", "noisy_process"]
"usr_ref" may be omitted
"proc_meta" may be omitted
```

rewritten

3. Observed drifted event after hard4

The resulting Backend B JSON is variant-dependent and is obtained by the Blue-team agent via an MCP-like call.

```
{
  "tstamp": "2026-02-19T13:50:01Z"
  "incident_case": 23,
  "seed_id": 22260,
  "kind": "proc_event"
  "node": "ws-02",
  "account": "bob",
  "ip_from": "10.0.10.22"
  "ip_to": null,
  "verb": "proc_spawn"
  "result_code": "OK",
  "priority": "P3"
  "image": "python",
  "labels": ["BENIGN", "NOISY_PROCESS"]
}
```

In this example, the **v3_semantic** variant is selected

raw drifted event

4. Schema normalizer: Infer Backend B canonical field mapping

Infer or reuse field mapping as the agent does not consume Backend B raw keys directly

Five-step resolution chain:

1. Run-local cache
2. Shared signature cache
3. Static contract-first aliases
4. LLM normalizer
5. Confidence gate

Sources: MCP aliases, cache, optional LLM mapping

Mapping

5. Normalized event

Normalized output follows the 13-field canonical contract

- timestamp
- event_type
- host
- src_ip
- dst_ip
- action
- severity
- tags
- user
- outcome
- process_name
- episode_id
- seed

Canonical record emitted

6. Blue-team analysis continues

The agent sees canonical fields after schema normalization.

*enrich -> retrieve memory -> correlate
-> decide -> simulated gate/act -> log*